

Quality of AI-Generated vs. Human-Generated Code

Tasneem Muhammed Eltabakh

Communication & Information Engineering
Zewail City
Giza, Egypt
s-tasneem.attia@zewailcity.edu.eg

Nada Nabil Soudi

Communication & Information Engineering
Zewail City
Giza, Egypt
s-nada.soudi@zewailcity.edu.eg

Doaa Shawky

Software Development Program
Zewail City
Giza, Egypt
dshawky@zewailcity.edu.eg

Abstract—This study examines the quality differences between AI-generated and human-generated code through an evaluation of multiple software quality metrics, including maintainability, complexity, and documentation. Using a dataset of 5,312 code samples—2,700 human-generated and 2,612 AI-generated—we applied machine learning techniques to classify and analyze the code based on these metrics. The results revealed that AI-generated code tends to excel in maintainability and documentation, demonstrating higher maintainability index scores and a higher ratio of comments. Additionally, AI-generated code often features simpler control structures, reflected in its lower cyclomatic complexity. In contrast, human-generated code showcased greater adaptability and flexibility, particularly in addressing complex problem statements. A neural network classifier achieved 88.05% accuracy in distinguishing between the two code origins, with comments ratio, maintainability index, and cyclomatic complexity being the most significant differentiators. These findings highlight the complementary roles of AI and human contributions in software development, suggesting strategic integration of both for enhanced efficiency and quality.

Index Terms—AI-Generated Code, Human-Generated Code, Code Quality, LLMs, Software Quality Metrics

I. INTRODUCTION

The increasing integration of AI in software development raises a concern about the quality of AI-generated code compared to human-generated code. As AI continues to evolve, its ability to generate code has become a topic of growing interest within both academic and industrial fields.

In this study, we aim to evaluate and compare the quality of AI-generated and human-written code through an analysis using various software quality metrics. These metrics are commonly employed to assess code quality such as maintainability, complexity, comment ratio, and more.

The primary research question of this study is: *Do AI-generated and human-generated code differ in quality, and if so, is one superior to the other, or does the quality depend on other factors?*

To address this question, we build a dataset comprising both human-written code and AI-generated code, analyze their quality using multiple Software metrics, and apply machine learning techniques to classify the code based on the identified metrics.

This study seeks to explore insights into the strengths and weaknesses of AI-generated code versus human-generated

code, contributing to the broader understanding of AI's role in software development.

II. LITERATURE REVIEW

A. Overview

This literature survey examines the current state of research in code quality assessment, AI-generated code, and human-generated code, highlighting the gaps that our study aims to address. The review focuses on comparing AI-generated and human-written code, identifying quality metrics, and uncovering any gaps in current datasets and methodologies. To ensure the review is up-to-date, we have focused on papers published no earlier than 2022.

B. Current state of the Art

Recent work has explored the ability of AI-generated content detectors to identify AI-generated code. One study tested 13 large datasets containing 5,069 samples, finding that AIGC detectors struggle with larger datasets, leading to a significant drop in detection accuracy. The study concluded that the performance of these detectors decreases as dataset size increases, so it indicates a limitation when larger datasets are used. [1].

Another study explored the accuracy of distinguishing AI-generated code from human-written code using a small but diverse dataset. The results showed up to 0.92 accuracy in distinguishing between the two. However, the small sample size and lack of diversity limit the findings, highlighting the need for further research using larger and more varied datasets [2].

One study took a deeper look at the accuracy of code generated by AI, focusing on whether the code truly works as expected when faced with more complex challenges. The researchers developed EvalPlus, a tool designed to improve upon the current HumanEval benchmark by adding a larger variety of test cases. Through their work, they discovered that earlier assessments had been too optimistic about the AI's ability to produce correct code, especially when dealing with harder problems. Although EvalPlus does a good job of checking if the code functions properly, it doesn't address other important aspects like how easy the code is to maintain or read. Additionally, it is not structured as questions and

answers, which makes it less suitable for directly comparing human-written and AI-generated code [3], [4].

Stylometric features have also been used to distinguish AI-generated code from human-written code, particularly for Python programming problems. However, the study's scope was limited to a small dataset and a single programming language, reducing the broader applicability of the results [5].

A separate investigation compared AI-generated code with human-written code using the HumanEval dataset. The findings conclude that while AI-generated code performed well in generating syntactically correct solutions, it struggled with any of the implicit requirements in the problem descriptions. This suggests that AI models lack the cognitive flexibility that humans use to infer unstated problem conditions. The study's focus on the HumanEval dataset which is also not structured as questions and answers. [6]

Research on the security and plagiarism risks associated with AI-generated code highlights the vulnerabilities in such code when applied in sensitive domains. It was found that AI-generated code is often more uniform, making it easier to detect but also more prone to security issues, as it tends to use common patterns that attackers could exploit. This study advocates for the development of more secure AI coding models [7].

C. Gaps in Current Research

While existing research has made significant contributions to our understanding of code quality, several gaps and limitations need to be addressed:

- The lack of datasets where AI-generated and human-written code addresses the same questions, which is essential for a direct comparison of quality.
- The lack of studies that focus on assessing the quality of the code itself, rather than just determining whether it was written by AI or a human, using a wide range of metrics.
- The limited exploration of factors influencing code quality in both AI-generated and human-written code across various programming languages and problem domains.
- The absence of machine learning models that can effectively classify and distinguish between AI-generated and human-written code based on quality metrics.

D. Proposed Approach

In response to the identified gaps in current research, our study aims to provide a comprehensive analysis of the quality differences between AI-generated and human-written code. Our approach is characterized by the following key aspects:

- Creation of a new dataset where AI and human-generated code address the same problems, enabling a direct comparison of both.
- Evaluation of code quality using a wider variety of metrics, including the most well-known and widely accepted ones.

- Application of machine learning techniques to classify AI-generated versus human-written code based on a variety of quality metrics.

III. DATA SET

A. Human Dataset Collection

The human-written code was sourced from archives of platforms like LeetCode, Codeforces, and HackerRank via the Wayback Machine, focusing on pre-2014 data to ensure that the code was written without the influence of AI tools. Although this scraping yielded fewer examples than targeted, we supplemented the dataset with solutions from the dataset described in [1]. This dataset contains over 3.3k Python human coding problems and their corresponding solutions, including those involving strings, lists, arrays, tuples, dictionaries, and more advanced topics such as file I/O, loops, conditionals, object-oriented programming, and data structures and algorithms.

To ensure the accuracy of our dataset, we cleaned and validated all solutions, excluding incomplete or problematic code. After cleaning, 2,700 human-generated code samples were finalized. We further contacted the authors of the dataset to confirm that the code was indeed generated by humans and scraped before AI-generated code was widely available.

B. AI-Generated Code Collection

To generate AI solutions, we subscribed to the ChatGPT-4 API and used it to generate code for the same set of problems as found in the human-generated dataset. The AI was prompted with each problem, and the code it produced was collected to form the AI-generated portion of the dataset.

C. Main dataset description

The whole dataset is about 5,312 code samples, consisting of 2,700 human-generated solutions and 2,612 AI-generated solutions.

The entire dataset was labeled with a categorical variable indicating the origin of the code, where 0 denotes human-written code and 1 denotes AI-generated code. Each code sample was analyzed and labeled with several software quality metrics, including:

- **Cyclomatic Complexity:**

$$M = E - N + 2P \quad (1)$$

Measures the complexity of a program's control flow. E represents the number of edges, N is the number of nodes, and P refers to the number of connected components in the graph. A higher value indicates more complex code, requiring more testing paths [8].

- **Halstead Metrics [9]:**

– *Vocabulary:*

$$h = h_1 + h_2 \quad (2)$$

Represents the total number of unique operators (h_1) and operands (h_2) used in the program.

- *Length*:

$$N = N_1 + N_2 \quad (3)$$

Denotes the total occurrences of operators (N_1) and operands (N_2) in the program.

- *Volume*:

$$V = N \log_2 h \quad (4)$$

Represents the size of the implementation in bits. A larger volume suggests that the program is more difficult to understand.

- *Difficulty*:

$$D = \frac{h_1}{2} \cdot \frac{N_2}{h_2} \quad (5)$$

Measures the difficulty of understanding and implementing the code.

- *Effort*:

$$E = V \cdot D \quad (6)$$

Represents the total mental effort required to write or maintain the program.

- *Bugs*:

$$B = \frac{V}{3000} \quad (7)$$

Estimates the number of defects in the code.

- **Maintainability Index:**

$$MI = 171 - 5.2 \cdot \ln(\text{Volume}) - 0.23 \cdot \text{Cyclomatic Complexity} - 16.2 \cdot \ln(\text{SLOC}) + 50 \cdot \sin(\sqrt{2.46 \cdot \text{Comment Ratio}}) \quad (8)$$

Predicts the maintainability of the software. Higher values indicate easier maintenance [10].

- **Source Lines of Code (SLOC):** Simple count of the number of lines of code in the program. Measures the size of the codebase [11].
- **Comments Ratio:**

$$\text{Comments Ratio} = \frac{\text{Number of Comments}}{\text{SLOC}} \quad (9)$$

Indicates the level of documentation within the code. A higher ratio suggests better code documentation [12].

- **Number of Parameters:** Count of the number of parameters passed to a function, which can impact readability and complexity [13].
- **Afferent Coupling:** Measures the number of classes or components that depend on a given class. High afferent coupling may indicate that changes to the class could have widespread effects [14].
- **Efferent Coupling:** Counts the number of classes or components a given class depends on. High efferent coupling may indicate a class relies on many external dependencies [14].
- **Instability:**

$$I = \frac{\text{Efferent Coupling}}{\text{Afferent Coupling} + \text{Efferent Coupling}} \quad (10)$$

Represents the likelihood of a class or component to change. Values closer to 1 indicate high instability (high dependency on other modules) [15].

IV. METHODOLOGY

The methodology for this study is summarized in Fig. ?? . It provides an overview of the data collection, preprocessing, and analysis steps undertaken to compare AI-generated and human-generated code.

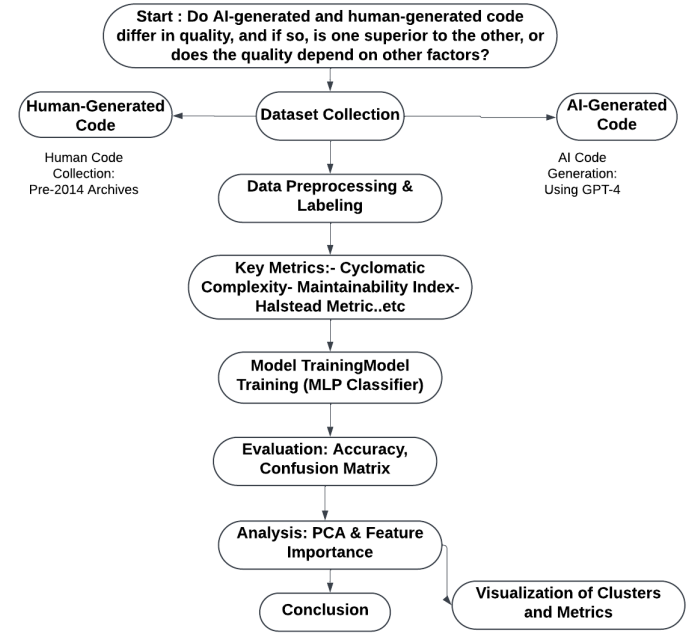


Fig. 1. Methodology workflow

A. Data Preprocessing

The dataset used in this study contains 5,312 code samples, with 2,700 human-generated code samples and 2,612 AI-generated code samples. The preprocessing steps involved the following actions:

- **Feature Selection and Data Cleaning:** We started by loading the dataset and dropping irrelevant columns such as 'Filename' and 'Syntax Errors.' This was necessary to ensure the features directly contributed to code classification based on quality metrics.
- **Label Separation:** The 'Label' column, which indicates whether a code sample is human-generated (0) or AI-generated (1), was separated from the feature set. The remaining columns, which consist of various software quality metrics, were used as input features.
- **Data Splitting:** The dataset was split into training and testing sets with an 80/20 ratio using the This ensured that the models could be evaluated on unseen data.

- **Feature Normalization:** We normalized the features to scale all features to have a mean of 0 and a standard deviation of 1. This step is very important to make sure that all features are on the same scale, especially for algorithms like neural networks that are sensitive to the magnitude of input features.

B. Model Selection and Training

We evaluated multiple machine learning classifier models to determine the best fit for classifying AI-generated and human-generated code based on the extracted software quality metrics. After experimenting with several models including Random Forest, Support Vector Machines (SVM), and XGBoost, the best-performing model was found to be a **Neural Network**, specifically a **Multi-Layer Perceptron (MLP) Classifier**, with 88.05%. The selected MLP model had the following configuration:

- Hidden layer sizes: 30 neurons
- Activation function: tanh
- Alpha: 0.0001 (L2 regularization term)
- Maximum iterations: 1000

C. Model Evaluation

The performance of the model was evaluated using multiple metrics:

- **Accuracy:** The accuracy of the model on the test set was 88.05%, indicating a high level of performance.
- **Confusion Matrix:** The confusion matrix (as shown in Table 1) shows that the model correctly classified most of the samples, with only a few misclassifications.

TABLE I
CONFUSION MATRIX

	Predicted Label 0	Predicted Label 1
Actual Label 0	547	4
Actual Label 1	2	510

- **Classification Results:** The classification report provided detailed precision, recall, and F1-scores for both human-generated and AI-generated code classifications.

D. Feature Importance Analysis

To understand the importance of each feature in classifying AI-generated and human-generated code, and to decide whether this feature is effective in differentiating between AI and Human codes we calculated the feature importance scores. As shown in Fig. 2, the top features that mainly differentiate between both are:

- **Comments Ratio**
- **Maintainability Index**
- **Cyclomatic Complexity**

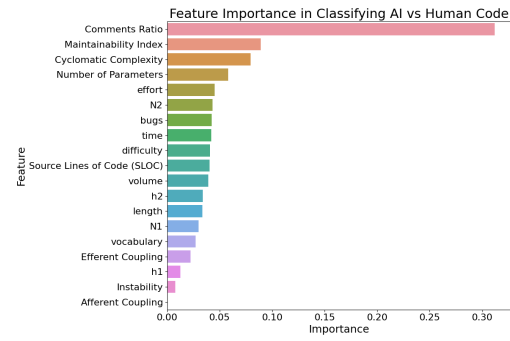


Fig. 2. Feature Importance in Classifying AI vs Human Code.

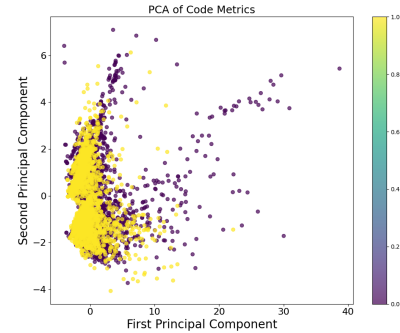


Fig. 3. PCA of Code Metrics.

E. Principal Component Analysis (PCA)

Principal Component Analysis was applied to reduce the dimensionality of the feature space and visualize the separation between AI-generated and human-generated code. The PCA plot in Fig. 3 shows that the two classes were well separated.

The primary research question of this study was: *Do AI-generated and human-generated code differ in quality, and if so, is one superior to the other, or does the quality depend on other factors?* Based on the analysis and visualizations, the following conclusions can be drawn:

- **Comments and Documentation:** AI-generated code contains significantly more comments (as shown in Fig. 4), indicating that it is better documented than human-generated code. This suggests that AI tools, such as ChatGPT-4, are designed to provide detailed explanations and context, enhancing the readability and comprehension of code.
- **Maintainability Index:** Human-generated code tends to score lower on the maintainability index compared to AI-generated code (as shown in Fig. 5). This suggests that AI-generated code might have a higher maintainability due to its more structured nature.
- **Cyclomatic Complexity:** AI-generated code tends to have lower cyclomatic complexity, which is evident in Fig. 6. This indicates that AI models tend to generate simpler control structures, making the code more straightforward but possibly less flexible for handling complex

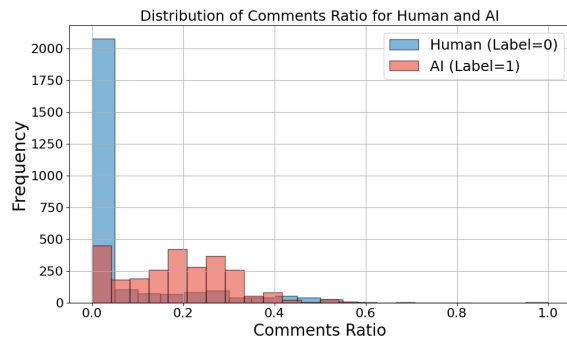


Fig. 4. Distribution of Comments Ratio for Human and AI code

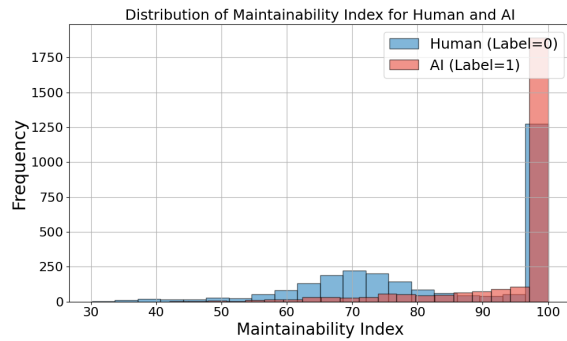


Fig. 5. Distribution of Maintainability Index for Human and AI code

logic.

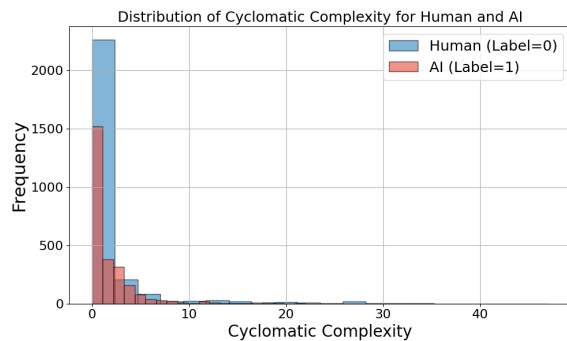


Fig. 6. Distribution of Cyclomatic Complexity for Human and AI code

These conclusions illustrate that the answer to the research question is **yes**, there are measurable differences in the quality of AI-generated and human-generated code. While AI-generated code tends to excel in maintainability and documentation, it may be more simplistic compared to human-written code. Further research is required to explore these differences across other programming languages and to evaluate the dynamic performance of AI-generated code in real-world applications

The code and datasets used in this research are available in the GitHub repository [16].

While this study provides valuable insights into the quality differences between AI-generated and human-generated code, several areas warrant further investigation. Future work could address the following limitations:

- **Multiple Programming Languages:** The dataset in this study is limited to Python code. Expanding the dataset to include other widely-used programming languages, such as Java, C++, or JavaScript, could provide a more comprehensive understanding of how AI-generated and human-generated code compare across different languages. These languages may exhibit unique characteristics in terms of maintainability, complexity, and other software quality metrics.
- **Human Dataset Source:** The human-generated code used in this study was sourced from third-party archives, and we did not directly scrape the data ourselves. In future research, a more controlled collection process could ensure greater consistency and remove uncertainties regarding the conditions under which the human code samples were collected.
- **Dataset Size and Diversity:** Although the current dataset contains over 5,000 samples, increasing the size and diversity of the dataset would help capture a wider spectrum of code quality variations, especially for more complex and diverse problem domains. Including more real-world problems from varied domains could yield deeper insights.
- **Dynamic Code Quality Metrics:** The current study focuses on static code analysis, such as maintainability and complexity metrics. Incorporating dynamic code metrics, such as runtime performance, memory efficiency, or security vulnerabilities, could provide a more holistic view of the differences between AI-generated and human-written code.
- **Different AI Models:** The AI-generated code in this study was produced solely by *ChatGPT-4*. Future research could explore code generated by different language models (e.g., Codex, GPT-3, or other models) to examine whether the findings are consistent across different AI systems or if certain models produce superior or inferior code based on quality metrics.

V. CONCLUSION

This research provides a deeper understanding of the unique strengths and limitations of AI-generated and human-generated code. AI-generated code stands out for its excellent maintainability and thorough documentation, making it highly suitable for tasks where structure and clarity are essential. On the other hand, human-written code remains unmatched in its ability to tackle complex and nuanced problems, reflecting the creativity and flexibility of human developers.

These findings highlight the complementary nature of AI and human contributions to software development. Rather than framing them as competitors, we see them as tools that can be strategically employed based on the requirements of a given project. For straightforward and repetitive tasks, AI

can save time and improve consistency. Meanwhile, humans bring critical thinking and adaptability to areas where abstract reasoning and deeper contextual understanding are needed.

Looking ahead, there is significant potential for further exploration in this field. Expanding the analysis to include diverse programming languages and real-world datasets will enhance our understanding of code quality across various domains. Additionally, incorporating dynamic metrics, such as runtime performance and security, will offer a more comprehensive picture of AI-generated code's capabilities. Exploring how different AI models perform on these metrics could further refine our understanding of their applications.

Ultimately, this study serves as a step toward bridging the gap between AI capabilities and human expertise, encouraging developers to make informed choices in integrating AI tools into their workflows. By understanding their respective strengths, we can leverage both AI and human ingenuity to create more efficient, innovative, and high-quality software systems.

ACKNOWLEDGMENT

We would like to express our gratitude to Jandrik Lana for providing us with the human dataset and for the detailed clarifications, he replied to our inquiries and ensured that the dataset was exclusively composed of human-generated code. Their assistance was invaluable to the success of this study.

REFERENCES

- [1] W. H. Pan, M. J. Chok, J. L. Shen, and X. K. Ong, "Assessing AI Detectors in Identifying AI-Generated Code," *Proceedings of ACM Conference (Conference'17)*, 2024.
- [2] F. Bukhari, B. Tan, and L. De Carli, "Distinguishing AI- and Human-Generated Code: A Comparative Study," in *2023 Workshop on Software Supply Chain Offensive Tactics (SSCOT)*, 2023.
- [3] "EvalPlus: Analyzing AI-Generated Code," GitHub, 2024. [Online]. Available: <https://github.com/evalplus/evalplus>
- [4] "HumanEval: Evaluating AI Code Generation," GitHub, 2024. [Online]. Available: <https://github.com/openai/human-eval>
- [5] O. J. Idialu, N. S. Mathews, and R. Sriwattanathamma, "Classifying Code as Human Authored or GPT-4 Generated: A Stylometric Approach," *MSR '24: Proceedings of the 2024 IEEE/ACM 21st International Conference on Mining Software Repositories*, 2024.
- [6] S. Akkus, "Can AI code like a human: A Critical analysis on HumanEval dataset," *Graduate School of Electrical Engineering*, 2024.
- [7] S. H. Ambati, "Security and Authenticity of AI-generated Code," *University of Saskatchewan Thesis*, 2023.
- [8] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308-320, 1976.
- [9] M. H. Halstead, "Elements of Software Science," *Elsevier*, vol. 7, no. 5, 1977.
- [10] P. Oman and J. Hagemester, "Metrics for Assessing a Software System's Maintainability," *Software Maintenance*, vol. 3, no. 2, pp. 337-344, 1994.
- [11] B. W. Boehm, "Software Engineering Economics," *Prentice-Hall*, 1981.
- [12] M. Gutierrez and D. Oriol, "The Impact of Commenting on Code Quality," *Empirical Software Engineering*, vol. 20, pp. 98-120, 2011.
- [13] S. R. Chidamber and C. F. Kemerer, "A Metrics Suite for Object-Oriented Design," *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476-493, 1994.
- [14] R. C. Martin, "Agile Software Development, Principles, Patterns, and Practices," *Prentice-Hall*, 2003.
- [15] N. Nabil and T. Eltabakh, "Quality-of-AI-Generated-vs.-Human-Generated-Code Dataset," GitHub, 2024. [Online]. Available: <https://github.com/TasneemEltabakh/Quality-of-AI-Generated-vs.-Human-Generated-Code/blob/main/main-dataset.xlsx>
- [16] N. Nabil and T. Eltabakh, "Quality-of-AI-Generated-vs.-Human-Generated-Code," GitHub, 2024. [Online]. Available: <https://github.com/TasneemEltabakh/Quality-of-AI-Generated-vs.-Human-Generated-Code.git>